

## Part A: (Understanding of Theory Concept)

### Q1) Explain Polymorphism in Java.

Write the definition of Polymorphism and explain why it is important in Object-Oriented Programming.

#### Answer-

Polymorphism is an important concept of Object-Oriented Programming (OOP). The word polymorphism means “many forms.” In Java, it allows the same method name or interface to perform different actions depending on the situation. In simple words, polymorphism means one name, multiple behaviors.

#### Why Polymorphism is Important

Polymorphism makes programs flexible, reusable, and easier to maintain. It reduces code duplication and allows the same interface to be used for different implementations. It also makes it easier to extend a program without changing existing code.

#### Real-Life Analogy

A person may behave differently in different situations—for example, as a student, employee, or customer. Similarly, a Java method can perform different tasks depending on the arguments or the object calling it.

### Q2) Explain the Types of Polymorphism

Explain the following types of Polymorphism with suitable examples:

- Compile-Time Polymorphism (Static Polymorphism)
- Run-Time Polymorphism (Dynamic Polymorphism)

#### Answer-

#### A. Compile-Time Polymorphism (Static Polymorphism)

Compile-time polymorphism is achieved through method overloading. The compiler decides which method should be executed during compilation. The methods have the same name but different parameter lists.

#### Example

```

class Calculator {
    int add(int a, int b) { return a + b; };
    int add(int a, int b, int c) { return a + b + c; };
    double add(double a, double b) { return a + b; };
}

public static void main(String[] args) {
    Calculator c = new Calculator();
    System.out.println(c.add(10, 20));
    System.out.println(c.add(10, 20, 30));
    System.out.println(c.add(5.5, 4.5));
}
}

```

Here, add() has the same name but different numbers or types of parameters. The compiler selects the correct method based on the arguments.

## B. Run-Time Polymorphism (Dynamic Polymorphism)

Run-time polymorphism is achieved through method overriding. It occurs when a child class provides its own implementation of a method already defined in the parent class. The method to be executed is decided during program execution.

### Example

```

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}

```

Although the reference variable is of type Animal, the object is a Dog. Therefore, the Dog class's sound() method executes at runtime.

### Q3) Explain Method Overloading

Your explanation should include:

- Definition of Method Overloading
- Rules of Method Overloading
- Different ways to overload a method
- Why changing only the return type is not considered Method Overloading
- Difference between Method Overloading and Method Overriding

#### Answer-

##### Definition

Method overloading is a Java feature in which two or more methods in the same class have the same name but different parameter lists. It is an example of compile-time polymorphism and allows similar operations to be performed using one method name with different inputs.

##### Example

```
class Display {  
    void show(int a) {  
        System.out.println("Integer: " + a);  
    }  
    void show(String name) {  
        System.out.println("String: " + name);  
    }  
}
```

##### Rules of Method Overloading

1. Methods must have the same name.
2. Their parameter lists must be different.
3. The return type may be the same or different.
4. Changing only the return type is not sufficient.
5. Overloading is resolved at compile time.

##### Different Ways to Overload a Method

1. By changing the number of parameters:

```
void display(int a)  
void display(int a, int b)
```

2. By changing the data type of parameters:

```
void display(int a)
```

```
void display(double a)
```

3. By changing the order of parameters:

```
void display(int a, String b)
```

```
void display(String b, int a)
```

Why Return Type Alone Does Not Overload a Method

Java identifies a method using its name and parameter list, not only its return type. Therefore, two methods with the same name and same parameters but different return types cause a compilation error. The compiler cannot decide which method to call based only on the return type.

Invalid Example

```
class Demo {  
    int show(int a) { return a; }  
    double show(int a) { return a; } // Compilation error  
}
```

Difference Between Method Overloading and Method Overriding

Method Overloading:

- Same method name with different parameters.
- Compile-time polymorphism.
- Usually occurs in the same class.
- Inheritance is not compulsory.

Method Overriding:

- Same method signature in parent and child classes.
  - Run-time polymorphism.
  - Requires inheritance.
  - The child class provides a specific implementation of the parent method.
-

## Part B: (Programming Questions)

### Q1) Calculator Addition

Create a class Calculator and overload the add() method to perform the following operations:

Add two integers

Add three integers

Add two decimal numbers.

Create an object of the class and call all overloaded methods.

```
class Calculator {  
    // Add two integers  
    int add(int a, int b) {  
        return a + b;  
    }  
    // Add three integers  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
    // Add two decimal numbers  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class partB_Q1 {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println("Sum of two integers: " + c.add(10, 20));  
        System.out.println("Sum of three integers: " + c.add(10, 20, 30));  
    }  
}
```

```
        System.out.println("Sum of two decimal numbers: " + c.add(10.5, 20.5));
    }
}
```

## **Output:**

A screenshot of a Java IDE's terminal window. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active, showing a PowerShell prompt. The user has entered the command 'java partB\_Q1.java'. The output of the program is displayed as follows: 'Sum of two integers: 30', 'Sum of three integers: 60', and 'Sum of two decimal numbers: 31.0'. The prompt is now ready for the next command.

```
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8> java partB_Q1.java
Sum of two integers: 30
Sum of three integers: 60
Sum of two decimal numbers: 31.0
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8> 
```

## **Q2) Display Method**

Create a class Display and overload the show() method to display student information in different formats:

Display only the student's name.

Display the student's name and age.

Display the student's name, age, and marks.

Create an object and call all versions of the show() method.

```
class Display {
    // Display only name
    void show(String name) {
        System.out.println("Name: " + name);
    }

    // Display name and age
    void show(String name, int age) {
        System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
    }

    // Display name, age and marks
    void show(String name, int age, double marks) {

        System.out.println("Name: " + name);

        System.out.println("Age: " + age);

        System.out.println("Marks: " + marks);

    }
}

public class partB_Q2 {

    public static void main(String[] args) {

        Display d = new Display();

        d.show("Harsh");

        System.out.println();

        d.show("Harsh", 18);

        System.out.println();

        d.show("Harsh", 18, 85.5);

    }

}
```

## **Output:**



The screenshot shows a PowerShell terminal window with the following content:

```
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8> java partB_Q2.java
Name: Harsh

Name: Harsh
Age: 18

Name: Harsh
Age: 18
Marks: 85.5
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8>
```

The terminal window has a title bar with "powershell" and standard window controls. The output of the Java program is displayed line by line, matching the expected output from the code above.

### Q3) Find Maximum

Create a class and overload the max() method to find:

Maximum between two integers.

Maximum between three integers.

Maximum between two decimal numbers.

Display the result for each method call.

```
class Maximum {  
    // Maximum between two integers  
    int max(int a, int b) {  
        return (a > b) ? a : b;  
    }  
    // Maximum between three integers  
    int max(int a, int b, int c) {  
        if (a > b && a > c)  
            return a;  
        else if (b > c)  
            return b;  
        else  
            return c;  
    }  
    // Maximum between two decimal numbers  
    double max(double a, double b) {  
        if (a > b)  
            return a;  
        else
```

```
        return b;
    }
}

public class partB_Q3 {

    public static void main(String[] args) {

        Maximum m = new Maximum();

        System.out.println("Maximum of two integers: " + m.max(10, 20));

        System.out.println("Maximum of three integers: " + m.max(10, 30, 20));

        System.out.println("Maximum of two decimal numbers: " + m.max(10.5, 20.8));

    }

}
```

### **Output:**



The screenshot shows a PowerShell terminal window with the following content:

```
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8> java partB_Q3.java
Maximum of two integers: 20
Maximum of three integers: 30
Maximum of two decimal numbers: 20.8
PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_8> |
```

The terminal window has a title bar with "powershell" and standard window controls. The output of the Java program is displayed line by line.